INTERNSHIP TASK REPORT

Name : **Remsha Imtiaz**	Student ID : **CA/DF1//33484**
Website : **CODE ALPHA**	Domain : **Cyber Security**

Task : 03

Secure Coding Review

Objective:

This task's goals are to look for security flaws in a Python-based login system, find typical ones such hardcoded credentials and a lack of input validation, and offer suggestions to guarantee secure development methods.

Introduction :

In order to find weaknesses and make recommendations for enhancements, this report offers a security analysis of a basic Python-based login system.

Language & Application :

- **Language:** Python
- **Application:** Login System

Python Code (Login.py) :

```
Username = input("Enter Your username : ")
Password = input("Enter Your password : ")
if Username == "Remsha Imtiaz" and Password == "0123" :
    print("Login Successful")
else:
    print ("Invalid Login")
```

Code Images:

```python
Username = input("Enter Your username : ")
Password = input("Enter Your password : ")
```

Figure 1 : Python login system code -part 1 showing user input section.

```python
if Username == "Remsha Imtiaz" and Password == "0123" :
    print("Login Successful")
else:
    print ("Invalid Login")
```

Figure 2 : Python login system code - part 2 showing login validation logic.

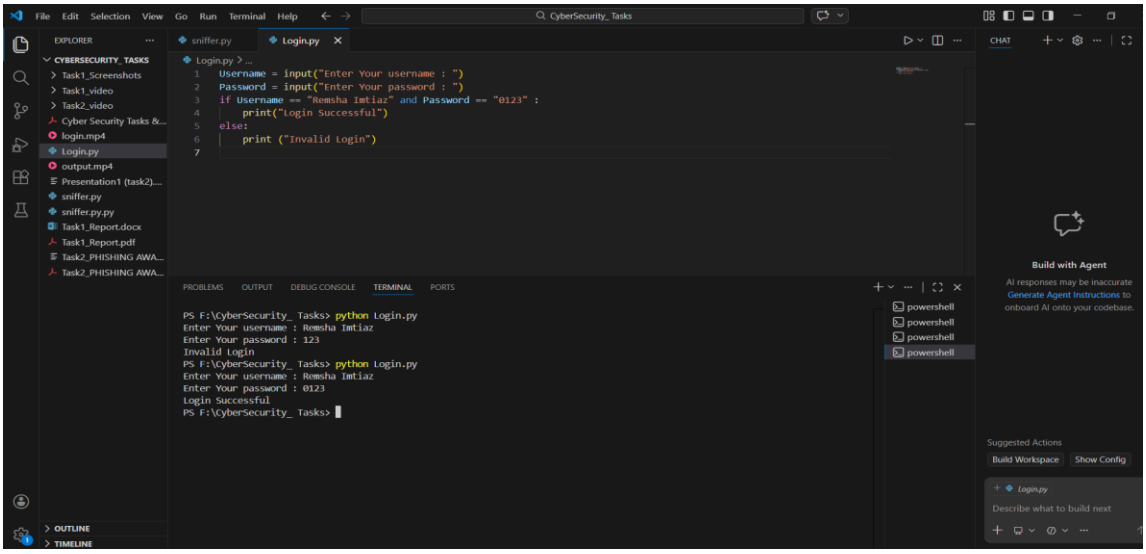

Figure 3 : Python login system code - part 3 showing success and failure output handling.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS F:\CyberSecurity_ Tasks> python Login.py
Enter Your username : Remsha Imtiaz
Enter Your password : 123
Invalid Login
PS F:\CyberSecurity_ Tasks> python Login.py
Enter Your username : Remsha Imtiaz
Enter Your password : 0123
Login Successful
PS F:\CyberSecurity_ Tasks>
```

Figure 4: Program output displaying login attempt result.

Identified Vulnerabilities :

- Hardcoded credentials
- No input validation
- Plain text password
- No login attempt limit

Risks :

These issues can allow unauthorized access and compromise system security.

Python Code (Secure _ Login.py) :

```
import hashlib
stored_username = "remsha imtiaz"
stored_password = hashlib.sha256("0123".encode()).hexdigest()
Username = input("Enter Your username: ").lower()
Password = input("Enter Your password: ")
hashed_password = hashlib.sha256(Password.encode()).hexdigest()
if Username == stored_username and hashed_password == stored_password:
    print("Login Successful")
else:
    print("Invalid Login")
```

Code Images:

```python
import hashlib

stored_username = "remsha imtiaz"
stored_password = hashlib.sha256("0123".encode()).hexdigest()

Username = input("Enter Your username: ").lower()
Password = input("Enter Your password: ")

hashed_password = hashlib.sha256(Password.encode()).hexdigest()
```

Figure 5 : Secure login system code - part 1 showing hashed password implementation.

```python
if Username == stored_username and hashed_password == stored_password:
    print("Login Successful")
else:
    print("Invalid Login")
```

Figure 6 : Secure login system code - part 2 showing secure login validation process.

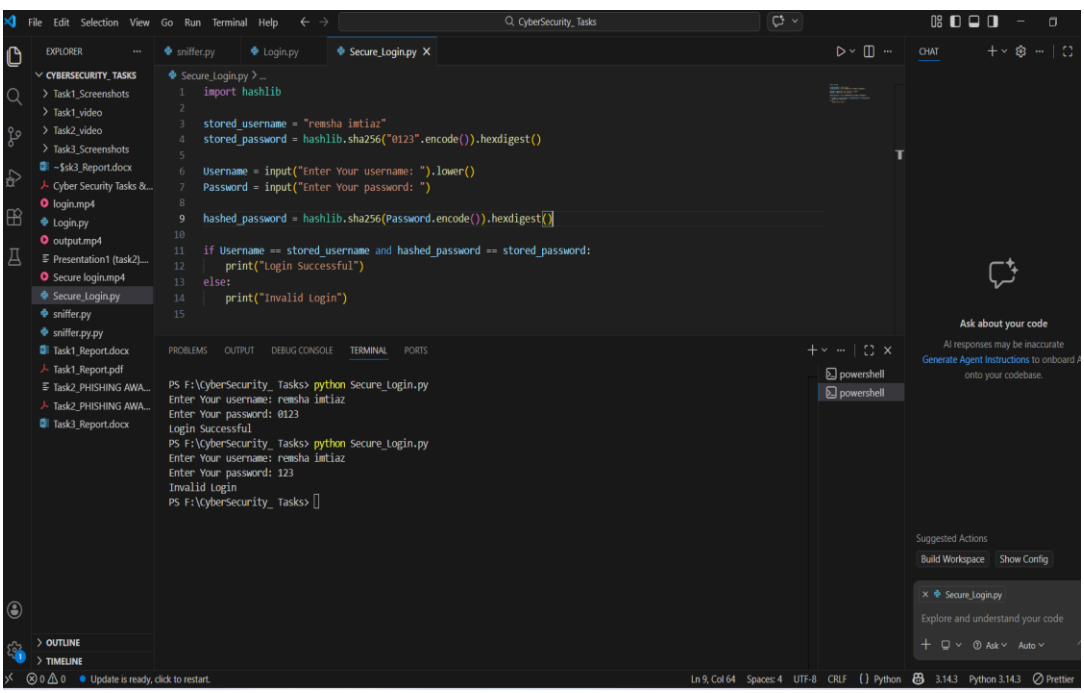

Figure 7 : Secure login system code - part 3 showing authentication and output handling.

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS F:\CyberSecurity_ Tasks> python Secure_Login.py
Enter Your username: remsha imtiaz
Enter Your password: 0123
Login Successful
PS F:\CyberSecurity_ Tasks> python Secure_Login.py
Enter Your username: remsha imtiaz
Enter Your password: 123
Invalid Login
PS F:\CyberSecurity_ Tasks>
```

Figure 8 : Output of the secure login system after implementing password hashing.

Explanation :

The internal implementation has been greatly enhanced even if the output of the secure and insecure versions looks the same. The safe version is more resilient to security attacks because it employs password hashing and handles user input better.

Comparison Between Login.py and Secure _ Login.py Code :

- The old login system was extremely susceptible to security risks since it kept passwords in plain text and used hardcoded credentials. The lack of encryption and input validation made it possible for hackers to get easy access to the system.

- To boost security, the upgraded version uses **sha-256** for password hashing. Additionally, user input is managed more securely, and private data is no longer kept in plain text. These enhancements greatly lower the possibility of unwanted access and strengthen the system's security.

Comparison Table :

Feature	Login.py Code	Secure_Login.py Code
Password Storage	Plain Text	Hashed
Security Level	Low	High
Input Handling	Weak	Improved
Risk Level	High	Reduced

Recommendations :

- Use hashed passwords
- Validate user input
- Avoid hardcoding credentials
- Limit login attempts

Conclusion :

The Python-based login system's **secure coding review** revealed a number of serious security flaws, such as hardcoded credentials, no input validation, and no encryption. These flaws could endanger the system's integrity and provide unwanted access. The application's overall security can be greatly increased by using secure coding techniques like password hashing, appropriate input validation, and avoiding hardcoded sensitive data. In order to guarantee reliable and secure systems, this activity highlights the significance of proactive vulnerability assessment and ongoing software development enhancement.